

The Stage-1 Diagnostic Patch

What it adds to `scan_engine.py`, why it cannot disturb a measurement, and how to read what it produces.

1. The question

Scans of the same spectral line, taken under nominally identical conditions, fall into groups. Within a group the curves lie on top of one another. Between groups they are shifted bodily along the wavelength axis by roughly 0.07 nm — same width, same height, same flank steepness, just translated. The configured Slip, the mechanical backlash figure entered in the GUI, is 0.080 nm.

A static test settled one thing already: at standstill nothing creeps. The signal does not wander after a move completes. That single observation rules out the averaging window as a cause, because measurement duration can only influence a recorded wavelength if something is moving while the measurement is taken. Nothing is. The correlation between the group boundary and the moment the Avg fraction was changed from 1 % to 50 % is therefore real but not causal — something else happened in that same gap.

What remains is a question the plot cannot answer:

Did the grating actually go somewhere different, or did the software simply give the same physical place a different name?

Both possibilities look identical on screen. Both move the peak. And both are equally invisible in the exported CSV, which records nothing but `index`, `wavelength_nm` and `voltage_V` — three columns that all derive from the software's own bookkeeping. There is no independent witness in the file.

Separating the two requires a second coordinate system, one the bookkeeping cannot influence. That is what this patch adds.

2. Why the encoder can serve as that second system

The obvious candidate is the drive's own position counter, `POS`. Whether it is usable depends entirely on whether anything ever resets it, so that is worth checking rather than assuming.

`init_motor()` sends `EN`, `V0`, optionally `ANSW`, and the ramp configuration. `recover_after_stop()` sends `ST`, `EN`, `V0`. Neither sends `H0`, and the string does not appear anywhere in the protocol layer. Nothing homes the drive.

`POS` is therefore a free-running absolute counter for the whole session, from Connect until the next power cycle. A scan does not reset it. Neither does a Stop, a recover, a Go To, or a Reference Run.

This is what makes the patch as small as it is. Raw `POS` values can be compared directly across every scan in a session, with no anchor arithmetic and no per-scan reference point. If two scans find the same peak at the same `pos`, the grating was in the same physical place both times — regardless of what wavelength the labels claim.

It helps to have the scale in mind while reading the logs:

Quantity	In motor steps
One nanometre	361 765
One scan step at 0.01 nm	3 618
One Slip (0.080 nm)	28 941
The observed offset (~0.07 nm)	~25 300
<code>POS_TOL_STEPS</code> , the arrival tolerance	90 ($\approx$ 0.25 pm)

The offset under investigation is about eight scan steps wide in encoder terms, set against an arrival tolerance of ninety steps. It sits far above the noise floor of the position system. If the cause is mechanical, it will not be subtle in `pos` — it will be glaring.

3. What the patch adds

Three hunks, all inside `scan_worker`, all in `scan_engine.py`. Nothing else in the codebase is touched.

3.1 One line per scan, taken at the anchor

The first hunk sits immediately after `grid_anchor_steps = read_position()` and records the state the scan is starting from:

```
[DIAG] scan-start anchor_pos=1234567 start_nm=587.5000 dir=+1
      last_move_direction=-1 slip_nm=0.080 src=NI-DAQ
```

The field that matters here is `last_move_direction`. It decides whether the jump to the scan's start position received backlash compensation, and it is mutated by every move the program makes — Go To, Reference Run, and Jog.

Jog deserves particular attention, because the GUI gives no hint of it:

```
def jog_plus_action():
    ...
    refs['entry_goto'].insert(0, f"{target:.3f}")
    goto_wavelength_action()          # a Jog IS a Go To
```

A jog is a complete Go To through the same code path, with the same `entry_current.get()` origin and the same call to `reversal_compensation_steps()`. One jog between two scans is enough to flip the direction state, so that the next scan is compensated where its predecessor was not. That is precisely the kind of event that would produce congruent groups with a step between them, and this line is what makes it visible — captured before the scan's own moves overwrite the value.

The `src` field records whether the voltages are real or simulated. Without an NI driver the simulator produces a convincing spectrum, complete with emission lines at 486, 587.5, 589 and 656 nm, which is exactly what makes it dangerous to mistake for data.

3.2 One line per measured point

The second hunk sits directly after the measurement, before the move to the next point:

```
[DIAG] i=12 label_nm=587.6200 pos=1276543 enc_nm=587.6198
      lbl_minus_enc_nm=+0.0002 anchor=1234567 slack=0
      dir=+1 reads=1 win_ms=4.5 V=2.31074
```

`label_nm` is what the software calls this point — the value that goes into the plot and the CSV. `pos` is the raw encoder count, and `enc_nm` is that same count expressed in nanometres relative to the scan's anchor. The difference between the two, `lbl_minus_enc_nm`, is the actual diagnostic: it is the amount by which the label and the physical position disagree.

The remaining fields provide context. `anchor` allows cross-scan arithmetic. `slack` shows the compensation steps folded into the position target but deliberately not into the label. `dir` is the direction state at that moment.

`reads` and `win_ms` are there to close out the averaging question with evidence rather than argument. At Timebase 10 ms and Pre-settle 1 ms the available window is 9 ms, while a single DAQ read takes around 29 ms. Both 1 % and 50 % therefore yield exactly one read. If `reads=1` appears in both groups, the two settings were bitwise identical and the correlation with the group boundary was coincidence — demonstrated, not asserted.

3.3 Removing a read that is now redundant

```
-          pos_nm_next = pos_nm + st_nm*direction
+          curr_abs = read_position()
+          # curr_abs was read right after the measurement above
```

The second hunk reads `POS` about twenty lines earlier than the original code did. Between the old and new positions there is no motor command of any kind — only the plot append, a log call, and a stop-flag check. The value is necessarily identical, so the original read is redundant and is removed rather than duplicated.

4. Why this cannot disturb a measurement

A diagnostic that changes the thing it measures is worse than no diagnostic, so this deserves more than a reassurance. Four checks, each mechanical rather than argued.

Serial traffic is unchanged. `read_position()` appears ten times in the file before the patch and ten times after. The call was moved, not added. This matters more than it might seem: the per-point time budget is already tight, and one extra round-trip per point would alter scan timing — exactly the sort of side effect that would contaminate the investigation.

No functions were added or removed. Confirmed by comparing the abstract syntax trees of the original and patched files.

Both new blocks are wrapped in `try / except Exception: pass`. A formatting error, a missing state key, or a `None` where a float was expected cannot interrupt a scan. The worst case is a log line that fails to appear.

No new imports. `ui_get`, `steps_to_delta_nm` and `daq_source_label` were already imported at the top of the file.

The patch applies cleanly to a pristine copy under `patch --dry-run`, and the result compiles.

There is one genuine side effect, small and arguably beneficial. If Stop is pressed during a measurement, `POS` is now read once before the stop check further down is reached — about ten milliseconds of delay on the abort path. In exchange, the last point actually measured gets its diagnostic line even when the scan is interrupted.

5. Reading the result

Pull the relevant lines out of the console log:

```
grep "\[DIAG\]" console.log > diag.txt
grep "\[SLIP\]" console.log > slip.txt
```

Then locate the peak of each scan and compare it in both coordinate systems. The interpretation is unambiguous:

What the data shows	What it means
<code>pos</code> at the peak is constant across scans while <code>label_nm</code> jumps	The grating is repeatable; the wavelength assignment is not. A bookkeeping fault.
<code>pos</code> at the peak jumps as well	The grating really does go elsewhere. Mechanics or drive; the label arithmetic is innocent.
<code>lbl_minus_enc_nm</code> grows during a single scan	Compensation is being applied mid-scan — unexpected, since direction never reverses within one.
<code>scan-start</code> shows a different <code>last_move_direction</code> between groups	The trigger is confirmed outright: one group's jump-to-start was compensated, the other's was not.
<code>reads=1</code> in both groups	The averaging change was inert. Correlation without causation, settled from data.

It is worth writing down in advance what the leading hypothesis predicts, so that the data can genuinely refute it rather than merely being consistent with it.

If the offset comes from compensation entering the position target but not the label, then `slack` will stay at zero for the entire scan — direction never reverses within a scan, so `reversal_compensation_steps()` returns zero on every step. A `[SLIP] Reversal` line will appear before one group's first scan and not before the other's, emitted by the jump-to-start rather than by the scan loop. `pos` at the peak will differ between groups by approximately 28 941 steps, one Slip. And `label_nm` at the peak will be identical between groups, because the label never saw the compensation.

If instead `pos` at the peak is the same in both groups while `label_nm` differs, the hypothesis is wrong and the cause lies upstream of the scan loop entirely.

6. What this patch does not cover

It instruments `scan_worker`, and nothing else. Two known defects live in code paths it does not touch, and neither will show up in the `[DIAG]` output.

The first is that **the stepped scan does not re-anchor after an abort**. When a step is interrupted, `scan_worker` breaks out of the loop without recovering the true position from `POS`, leaving `current_nm` and the Current λ field sitting at the last plotted point while the drive coasts past it along its deceleration ramp. `free_run_worker` performs exactly this recovery; `scan_worker` does not. The fix was evidently applied to one of the two paths and not the other.

The second is that **Go To continues driving after a Stop, working from a stale origin**. Following `[STOP] Motion interrupted`, `goto_worker` re-reads `entry_current.get()` — which was never updated — recomputes the delta from the original starting point, and issues a second move. On a long move it overshoots by roughly the distance already covered, and then records the target as the actual position. This is the reported disagreement between the mechanical dial and the software reading, and the error grows with the length of the move.

Both leave traces in the console log through lines that already exist, `[STOP] Scan interrupted` and `[STOP] Motion interrupted`, and both are better addressed by the manual tests in the accompanying report than by more instrumentation.

Some things were left alone deliberately. Nothing in the motion path was altered. The CSV format is untouched, as is the `PlotManager` structure. Adding encoder data to the exported CSV would mean changing what `PlotManager` stores, what `export_scan_csv_by_id` writes, and the file format itself — a large surface to disturb for a question that two scans' worth of log output will answer.

7. Removing the patch afterwards

The patch is self-contained and leaves no residue. Delete the `try / except Exception: pass` block that follows `grid_index = 0`, along with its comment. Delete the second such block following the `log(f"{pos_nm:.3f} nm | {v:.5f} V")` line, again with its comment. Then move `curr_abs = read_position()` back to its original place below `pos_nm_next = ...`, replacing the comment left there.

Every marker is greppable as `[DIAG]`, so nothing can be overlooked.

8. Sequence

One point of ordering matters, and it is easy to get wrong.

Do not repair the missing re-anchor before this data has been collected. Adding a re-anchor after an abort changes the very origin whose corruption one of the hypotheses is trying to demonstrate. Once that fix is in place, the hypothesis can no longer be tested. Measure first, then fix.

The sensible order is to apply this patch straight away — it is logging only, and can go in before any measurement. Then reproduce the offset with the averaging setting held fixed, using the GUI Jog buttons between scans: two baseline scans, a jog, two more scans, and one further scan to see whether the shift persists or springs back. Repeat the same sequence with the Slip field set to zero; if the offset disappears, the compensation arithmetic is the cause, and if it survives, it is not. Read the result against the table in Section 5. Only then write the fixes, for whichever defect the data actually implicates.